# Scientific Computing Exercise Set 2

Gileesa McCormack (13965751) & Alara Karadeniz (16384806) & Sooriyaa Karunaharan (16422805)

***Repository**: https://github.com/Gileesa/ScientificComputing*

## I. Introduction

PATTERN formation processes have fascinated scientists for over a century. From Turing's model of morphogenesis to Brownian trees, each process generates distinct patterns that can often be directly related to natural phenomena and the emergence of life. The development and implementation of numerical schemes to generate and study these patterns can ultimately help in understanding the emergence of complex structures found in natural phenomena.

In this paper, we implemented two well-known models used for numerical pattern formation: Diffusion-Limited Aggregation (DLA) and the Gray-Scott model. DLA is a form of stochastic pattern formation that creates complex spatial structures from local rules. The Gray-Scott model describes the interaction between two chemicals, which produce patterns such as stripes and spots, similar to those found in nature. In this paper, DLA was implemented using growth probabilities and Monte Carlo random walks, while the Gray-Scott reaction-diffusion system was simulated using a finite difference scheme. Our goal is to gain a better understanding of the strengths and limitations of these models.

## II. Theory

### A. Diffusion Limited Aggregation

Many systems in nature form complex branching structures due to diffusion-limited growth, e.g coral growth, mineral deposition, bacterial colonies, ... In such systems, particles or nutrients diffuse through a medium and attach to a growing cluster.

Diffusion-limited aggregation (DLA) is a mathematical model that describes such a growth process. In this model, structures emerge from the interplay between particle diffusion and irreversible aggregation.

The transport of particles is described by a concentration field $c(x,t)$, which represents the distribution of particles in the surrounding medium. The diffusive transport follows Fick's Law

$$J = -D\nabla c, \tag{1}$$

where $J$ denotes the particle flux, $D$ is the diffusion coefficient, and $\nabla c$ is the gradient of the concentration field. The rate of change in concentration can be expressed as

$$\frac{\partial c}{\partial t} = -\nabla \cdot J. \tag{2}$$

Substituting Fick's law into this relation yields the diffusion equation

$$\frac{\partial c}{\partial t} = -\nabla \cdot (-D\nabla c) \tag{3}$$

$$= D\nabla^2 c \tag{4}$$

Since the diffusion of the particles occurs on a much faster timescale than the growth of the cluster, the concentration field can be approximated as being in a steady state. Under this assumption, the time derivative of the concentration vanishes and the diffusion equation reduces to the Laplace equation

$$\nabla^2 c = 0. \tag{5}$$

To obtain the concentration field numerically, the Laplace equation is solved on a discrete lattice using the Successive Over-Relaxation (SOR) method. The concentration at each lattice site is iteratively updated using the average of its four neighboring cells. To accelerate convergence, the update rule is modified by introducing a parameter $\omega$,

$$c_{i,j}^{n+1} = \frac{\omega}{4}(c_{i+1,j}^{n} + c_{i-1,j}^{n+1} + c_{i,j+1}^{n} + c_{i,j-1}^{n+1}) + (1-\omega)c_{i,j}^{n}, \tag{6}$$

where $\omega$ controls the convergence behavior of the iterative solver.

The growth of the cluster is caused by the influx of particles that stick to its boundary. The growth velocity of the boundary is proportional to the incoming flux and can be written as

$$v_n = \frac{D}{\rho_0}\frac{\partial c}{\partial n}, \tag{7}$$

where $v_n$ is the velocity normal to the boundary, $\rho_0$ is the density of the particle, and $\frac{\partial c}{\partial n}$ denotes the concentration gradient in the normal direction of the boundary.

Although the concentration field is continuous, the DLA model is implemented on a discrete lattice, where

growth occurs at the individual lattice sites. The cluster can grow only at neighboring lattice sites that are adjacent to the existing cluster but not yet occupied.

The probability that a candidate site $(i, j)$ becomes part of the cluster depends on the local concentration and is given by

$$p_g(i,j) = \frac{c_{i,j}^{\eta}}{\sum \text{growth candidates } c_{i,j}^{\eta}}, \tag{8}$$

where $c_{i,j}$ denotes the concentration at the candidate site and $\eta$ controls how strongly the concentration field influences the growth process. At each growth step, a candidate site is randomly selected according to the probability defined in Equation 8 and added to the cluster. This stochastic selection ensures that sites with higher concentration values are more likely to be chosen, while still allowing growth at other candidate locations.

### B. Monte Carlo DLA

The Monte Carlo method is another way for simulating the DLA model. This method involves starting out with a single cell in the middle of the bottom boundary. Then releasing walkers from a random lattice point from the top boundary and letting them move in random directions each time step until they reach a lattice point that neighbors the cluster which at that point the cell becomes part of the cluster. The walker can move one lattice point at a time (up, down, right or left). If the walker ends up walking off the top or bottom boundaries, then the cell is removed from the system. If the cell moves to the left or the right boundary then it appears on the other side. Since there is no calculation of probabilities of spawning like in equation 8 then we can add the variable $p_s$, which represents the sticking probability of the walker to the cluster. If the walker does not stick to the cluster, then it keeps moving until it hits another neighbor without walking to an occupied location.

### C. Gray-Scott Model

The Gray-Scott model is a diffusion-reaction model that describes the reaction between two chemicals that also their diffusion. The system of reactions between chemicals $U$ and $V$ is as follows:

$$U + 2V \rightarrow 3V \tag{9}$$

$$V \rightarrow P \tag{10}$$

The reactions in Equation 9 and 10 describe how U and V react to create more V and how V decays into chemical P, respectively. Together with the diffusion of U and V in accordance with Fick's law, we can describe the change in concentrations of chemicals U and V as follows:

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u - uv^2 + f(1-u) \tag{11}$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + uv^2 - (f+k)v \tag{12}$$

Equation 11 describes the change in concentration of $U$. The three terms in the equation represent the diffusion term, the reaction term and replenishing term, respectively. Constant $f$ controls the rate of replenishing of $U$. In Equation 12, the change of concentration $V$ is described. The three terms here are the diffusion term, the reaction term and the decay term, respectively. Term $f + k$ denotes the rate at which $V$ decays. Since we are not interested in chemical $P$, we do not state an equation for it.

To find the numerical solution of this system of equations, we applied the method of lines. We first discretise the two dimensional space into an NxN grid. For the diffusion term, we can add up a forward and backward finite difference scheme to retrieve a 5-point numerical scheme. The finite difference schemes for the reaction and replenish/decay term are straightforward, as they are only dependent on one cell. This results in the following scheme for chemical U:

$$\begin{aligned}\frac{\partial u_{i,j}}{\partial t} = &\frac{D_u}{\delta x^2}\left(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}\right) \\ &- u_{i,j}v_{i,j}^2 + f(1-u_{i,j})\end{aligned} \tag{13}$$

This would yield a similar equation for chemical V.

Next, the temporal coordinate can be discretised. By applying the forward Euler method, we get the following two equations for U and V:

$$\begin{aligned}u_{i,j}^{k+1} = u_{i,j}^k + &\frac{\delta t D_u}{\delta x^2}\Big(u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k\Big) \\ &- \delta t\, u_{i,j}^k (v_{i,j}^k)^2 + \delta t\, f(1-u_{i,j}^k)\end{aligned} \tag{14}$$

$$\begin{aligned}v_{i,j}^{k+1} = v_{i,j}^k + &\frac{\delta t D_v}{\delta x^2}\Big(v_{i+1,j}^k + v_{i-1,j}^k \\ &+ v_{i,j+1}^k + v_{i,j-1}^k - 4v_{i,j}^k\Big) \\ &+ \delta t\, u_{i,j}^k (v_{i,j}^k)^2 - \delta t\,(f+k) v_{i,j}^k\end{aligned} \tag{15}$$

These numerical schemes can be used to update all cells except those at the boundary. For the boundary

cells, we applied Neumann boundary conditions, where the flux at the boundaries is zero. Through Von Neumann stability analysis on the diffusion term, we can find the following requirement for stability: $\frac{\delta t D}{\delta x^2} \leq 1$.

## III. Method

### A. Diffusion Limited Aggregation

In order to simulate the diffusion-limited aggregation process, the mathematical model described in the previous section is implemented numerically in a discrete computational domain. The computational domain is represented by a two-dimensional square lattice of size $N \times N$, where $N = 100$. Each lattice site corresponds to a spatial grid point where the concentration field is defined. The cluster is represented by a binary object matrix, where occupied lattice sites correspond to particles that have already been added to the cluster. The simulation is initialized with a single seed particle placed at the bottom of the computational domain.

Dirichlet boundary conditions are applied at the top and bottom boundaries of the domain. The concentration is fixed to $c = 0$ at the bottom boundary and $c = 1$ at the top boundary. For the left and right boundaries, Neumann boundary conditions are imposed to ensure zero flux through the boundaries. This is implemented numerically by setting the boundary values equal to those of their neighboring interior cells.

The concentration field is obtained by solving the Laplace equation using SOR method. In this project, the concentration field is iteratively updated for a maximum 1000 SOR iterations at each growth step. The cluster growth is simulated for 1000 growth steps.

During the iterative solution of the concentration field, numerical truncation errors can lead to small negative concentration values. Because concentration is a physical quantity that must remain non-negative, the values of $c$ are clipped to the interval $[0, 1]$ after each iteration. Since the negative values are extremely small and, therefore, negligible, clipping is applied to preserve both the physical meaning and numerical stability.

We also wanted to explore the possibility of speeding up the process. To do this we have implemented the SOR in a checkerboard pattern (red-black ordering) where each lattice point is colored black or red in such a way that no lattice point in the same color set affect each other, exactly like a checkerboard. This allows for a code where the lattice points within the same color set can be run simultaneously. This made for a more efficient code, which the results for will be discussed further in discussion. For the parallelization, we have used the library `Numba`. The performance times were measured using the python built in library `time`.

**Algorithm 1** One Growth Step for Laplace–DLA

**Input:** obj_matrix $\in \{0,1\}^{N\times N}$, concentration field $c$, parameters $\eta, \omega$
**Output:** updated obj_matrix
**(1) Solve** $\nabla^2 c = 0$ **with SOR until convergence:**
**repeat**
  **for** $i = 2$ **to** $N-1$ **do**
    **for** $j = 2$ **to** $N-1$ **do**
      **if** $\texttt{obj_matrix}_{i,j} = 1$ **then**
        $c_{i,j} \leftarrow 0$ ▷ absorbing cluster
      **else**
        $c_{i,j} \leftarrow (1-\omega)c_{i,j} + \frac{\omega}{4}(c_{i+1,j} + c_{i-1,j} + c_{i,j+1} + c_{i,j-1})$
      **end if**
    **end for**
  **end for**
  Apply boundary conditions; clip $c$ to $[0,1]$
**until** $\max|c - c_{\text{old}}| < \varepsilon$
**(2) Candidates:** $\mathcal{C} \leftarrow$ empty sites adjacent (N,E,S,W) to the cluster
**(3) Growth probabilities:** for each $(i,j) \in \mathcal{C}$ set $w_{i,j} \leftarrow (c_{i,j})^{\eta}$
$p_{i,j} \leftarrow \frac{w_{i,j}}{\sum_{(a,b)\in\mathcal{C}} w_{a,b}}$
**(4) Sample growth site:** draw one $(i,j) \in \mathcal{C}$ with probability $p_{i,j}$
$\text{obj_matrix}_{i,j} \leftarrow 1$
**return** obj_matrix

### B. Monte Carlo DLA

For implementing the Monte Carlo DLA we ran on a $100 \times 100$ board size with 50000 walkers. We chose such a high number of walkers to make sure that the clusters we obtained were of sufficient size for our results. For each time step we would randomly pick from a list of directions such as $(1,0), (0,1), (-1,0), (0,-1)$ to decide the direction the cell would move in. Once the cell reached a lattice point that neighbor the cluster then we generate a random float between 0 and 1 to determine if this float is less than $p_s$ to decide if the cell should stick to the cluster or move to the next location.

### C. Gray-Scott Model

To implement the numerical scheme described in Equations 14 and 15, the Python programming language was used with the `numpy` and `matplotlib` libraries.

To keep track of the concentrations of $U$ and $V$ in the two-dimensional space over time, two matrices were used. Each matrix element represents the concentration of either $U$ or $V$ at one spatial grid point in the $N$x$N$

space, where $N$ was set to 200, with cell sizes of $\delta x = \delta y = 1$.

The matrices of $U$ and $V$ at $t = 0$ were initialized. The $U$-matrix was set to a uniform concentration of 1. To initialize the $V$-matrix, the concentration of all elements was set to zero, except for a 10x10 square in the center, which had a base concentration of 0.25. Optionally, small random noise could be added to the square, slightly perturbing the concentration values. These values were clipped to be above zero to ensure no concentration could be negative.

The pseudocode for the update scheme for chemical $V$ is given in Algorithm 2. A similar algorithm is used for $U$, with only a difference in the reaction and the replenishment term.

**Algorithm 2** One Explicit Time Step for $V$ (Gray–Scott Model)

**Input:** Matrices $U^n, V^n$, parameters $D, f, k$, grid sizes $N_x, N_y$, $\Delta t, \Delta x$
**Output:** Updated matrix $V^{n+1}$
$\alpha \leftarrow \dfrac{D\Delta t}{\Delta x^2}$
**for** $i = 2$ **to** $N_x - 1$ **do**
  **for** $j = 2$ **to** $N_y - 1$ **do**
    $V_{\text{diffusion}} \leftarrow V^n_{i+1,j} + V^n_{i-1,j} + V^n_{i,j+1} + V^n_{i,j-1} - 4V^n_{i,j}$
    $V^{n+1}_{i,j} \leftarrow V^n_{i,j} + \alpha V_{\text{diffusion}} + \Delta t\, U^n_{i,j}(V^n_{i,j})^2 - \Delta t\,(f+k)V^n_{i,j}$
  **end for**
**end for**
$V^{n+1}_{1,j} \leftarrow V^{n+1}_{2,j}$ ▷ Top boundary (reflective)
$V^{n+1}_{N_x,j} \leftarrow V^{n+1}_{N_x-1,j}$ ▷ Bottom boundary (reflective)
$V^{n+1}_{i,1} \leftarrow V^{n+1}_{i,2}$ ▷ Left boundary (reflective)
$V^{n+1}_{i,N_y} \leftarrow V^{n+1}_{i,N_y-1}$ ▷ Right boundary (reflective)
**return** $V^{n+1}$

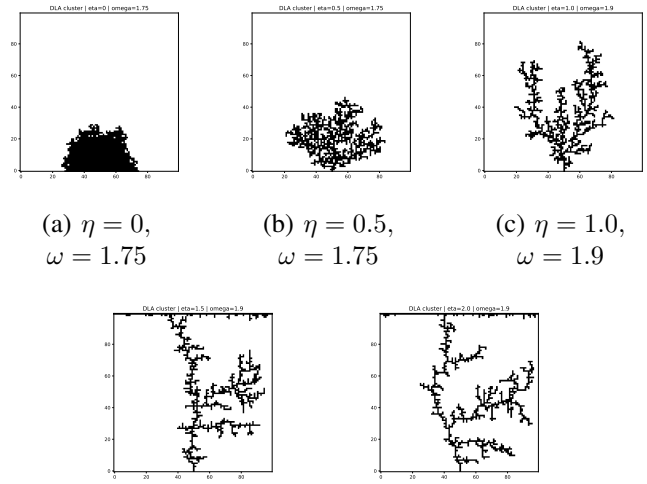
The updating schemes for $U$ and $V$ were ran for $N_t = 10000$ steps of size $\delta t = 1$ for different initial conditions of $D_u$, $D_v$, $f$ and $k$, resulting in different patterns. We did this for both non-noise and noise options for initialization of the $V$ matrix.

## IV. Results

### A. Diffusion Limited Aggregation

From Assignment 1, we know that the optimal SOR parameter typically lies in the interval $\omega \in [1.7, 2]$. Therefore, we explore the values of $\omega$ within this range together with $\eta \in [0, 2]$, which captures the region where the solver performs optimally. We tested $\eta \in 0, 0.5, 1, 1.5, 2$ and $\omega \in 1.75, 1.8, 1.85, 1.9, 1.95$.

For small values of $\eta = 0, 0.5$, lower $\omega = 1.75$ performs the best on average, while for larger $\eta \geq 1$, $\omega = 1.9$ gives the lowest mean iteration count 2a.




(a) $\eta = 0$, $\omega = 1.75$ (b) $\eta = 0.5$, $\omega = 1.75$ (c) $\eta = 1.0$, $\omega = 1.9$

(d) $\eta = 1.5$, $\omega = 1.9$ (e) $\eta = 2.0$, $\omega = 1.9$

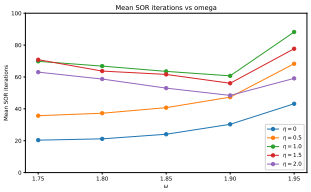
Fig. 1: Comparison of the DLA clusters obtained using the optimal $\omega$ for each tested value of $\eta$.

Figure 1 shows that the parameter $\eta$ strongly affects the morphology of the cluster. For $\eta = 0$, the cluster is compact and resembles an Eden like growth (see Figure 1a). At $\eta = 0.5$, the cluster remains relatively dense but develops a moderately branched structure (see Figure 1b). In contrast, for $\eta \geq 1$, the cluster becomes increasingly branched and sparse, with growth concentrated at a small number of active tips (see Figure 1d, e). This produces elongated structures that extend more rapidly toward the boundaries of the simulation domain. Moreover, for $\eta = 1$, the cluster exhibits a coral-like structure, with a large number of branches (see Figure 1c). Additionally, the differences between $\omega$ values also become more visible. Using small values e.g $\omega = 1.75, 1.8$ results in faster outward growth of the cluster, allowing the cluster to reach the grid boundary earlier than the optimal $\omega$ value.

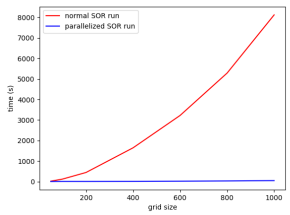
Furthermore, the animation of the growth process also reveals differences in how new particles are added to the cluster [2]. For small values of $\eta$, new particles attach at many locations along the cluster boundary, leading to growth in multiple directions. In contrast, for $\eta \geq 1$, growth occurs predominantly at the currently active branch tips.

For testing out the speed of our parallelized DLA, we tested out different board sizes to see how sequential SOR performs against parallelized SOR. Which can be seen visualized in the figure 2b. For these results

we wanted to run it multiple times to get mean and standard deviation values for more accurate results and error bars, but due to the runtime for this benchmark experimentation this would not have been possible to achieve.



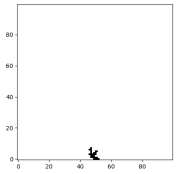

(a) Mean SOR iterations for different values of $\omega$ and $\eta$, showing how parameter choice affects convergence.



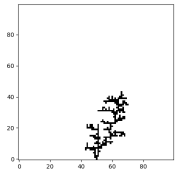

(b) Sequential vs. Parallel SOR performance times against different board sizes

### B. Monte Carlo DLA

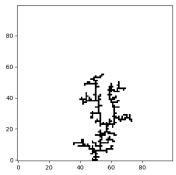
We ran our Monte Carl DLA implementation with different $p_s$ values to see how clusters get affected which we made several visuals with different $p_s$ values in figures 3. We are only including here figures for $p_s$ values 0.1, 0.6, and 1.0 but the rest of the figures can be found in our GitHub repository [3].



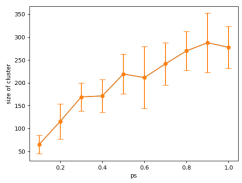
(a) $p_s = 0.1$



(b) $p_s = 0.6$



(c) $p_s = 1.0$



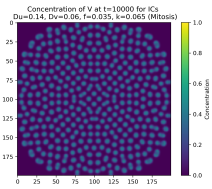

(d) $p_s$ values plotted against sizes of the clusters they create

Fig. 3: Different cluster that have formed through Monte Carlo DLA with different $p_s$ values and the size of clusters that have formed

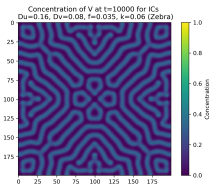
We also visualized the different cluster sizes against the $p_s$ values which can be seen in figure 3d. For this figure we have ran the simulation multiple times with the same parameters to get mean values for the sizes, which then we also got the standard deviations to visualize the error bars.

### C. Gray-Scott Model

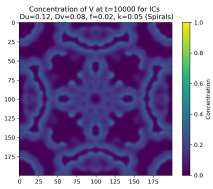
We ran the Gray-Scott simulation for different initial conditions of parameters $D_u$, $D_v$, $f$ and $k$, which created different patterns. The final concentration of chemical $V$ in two-dimensional space can be found in Figure 4. The patterns all show a clear symmetrical structure. The initial conditions affect the overall shape of the patterns, as well as their growth speed, as can be seen in gifs found in the GitHub repository [4].




(a) Mitosis: $D_u = 0.14,\ D_v = 0.06,\ f = 0.035,\ k = 0.065.$




(b) Zebra Fish: $D_u = 0.14,\ D_v = 0.08,\ f = 0.035,\ k = 0.060.$

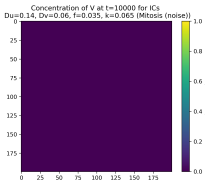



(c) Spirals: $D_u = 0.12,\ D_v = 0.08,\ f = 0.020,\ k = 0.050.$

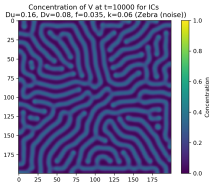
Fig. 4: Concentration of chemical $V$ of Gray-Scott simulation at t=10000 for different parameters.

Adding a noise term to the initialization of matrix $V$, we obtained results shown in Figure 5, with variation in the exact pattern due to the stochastic nature of the simulation. An animation for each Gray-Scott model plots in Figures 4 and 5 can be found in the GitHub repository [4].

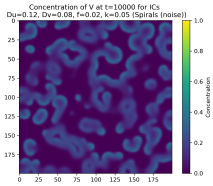
The patterns that included a noise term in the initialization of the $V$-matrix clearly show an asymmetric pattern. The growth speed does not seem affected, though the noise term can make the reaction die out if all concentration of V disappears, as can be seen in Figure 5a.




(a) Mitosis




(b) Zebra Fish




(c) Spirals

Fig. 5: Concentration of chemical $V$ in the Gray-Scott simulation at $t = 10000$ for different parameters. The initial $V$-matrix included random perturbation terms.

## V. Discussion

### A. Diffusion Limited Aggregation

The results demonstrate that parameter $\omega$ influences the numerical efficiency of the solver, while parameter

$\eta$ plays a key role in determining the morphology of the growing cluster. In Equation 8, the growth probability is proportional to the concentration raised to the power $\eta$, so increasing $\eta$ amplifies the differences in concentration along the boundary of the cluster. Consequently, sites with slightly higher concentration become much more likely to grow, while regions located in the inner parts of the cluster receive very little growth probability.

For small values of $\eta$, concentration differences along the boundary of the cluster are small, allowing particles to attach to many locations. This behavior produces compact clusters similar to Eden-type growth. As $\eta$ increases, growth becomes increasingly concentrated at exposed tips where the concentration is highest, leading to elongated branches and more sparse structures.

The results further show that the optimal value for the parameter $\omega$ depends on the cluster morphology. Compact growth, corresponding to low values of $\eta$, leads to more uniform changes in the Laplace field along the cluster boundary, making slightly smaller values of $\omega$ more efficient. In contrast, for larger $\eta$, growth occurs mainly at a few active tips, so the concentration field changes only locally between steps. Since the SOR solver is initialized with the solution from the previous step, the solver starts close to the new solution and converges rapidly, which explains why slightly larger values of $\omega$ perform better and why very low iteration counts are observed. This shows that the SOR parameter $\omega$ can still be optimized, but that its optimal value depends on the choice of $\eta$ [1].

When we look at our results from parallelization we can see a great improvement on runtime for the program compared to the sequential method. This was as expected as running multiple code in parallel saves quite a great deal of time.

### B. Monte Carlo DLA

When looking at our figures 3 for the Monte Carlo implementation we can see that as expected when we have a higher $p_s$ value, the clusters we achieve are larger in size since more cells stick to the cluster. We can also see in figure 3d, that generally there is a almost linear like relation between $p_S$ and cluster size. If we had more computational time to run with more walkers it is possible that we could see a better relationship between the two variables. When we look at the resulting clusters through the Monte Carlo method versus the regular diffusion model, we can see that the resulting trees are less complex patterns and are as spread out. This could be possibly explained by not using a diffusion equation but rather just letting random movements of the walkers decide the shape of the clusters.

### C. Gray-Scott Model

The results from the Gray-Scott reaction-diffusion system suggests that without perturbation in the initial concentration, symmetric patterns will form. With added perturbation, the symmetry of the shapes will be affected, though the time-scale of their growth remains the same. In addition, the perturbation can cause one of the chemicals to disappear, making the reaction die out.

## VI. Conclusion

In this paper we have explored different pattern formation processes. These include the Diffusion Limited Aggregation (DLA) mathematical model and implementation of it through the Monte Carlo method and also the Gray-Scott diffusion reaction model. Through our research we have looked into the effects of how changing $\eta$ in the DLA model affects the pattern formation and its effects on the optimal $\omega$. Which from we concluded that the optimal $\omega$ depends on the cluster morphology. We also looked into using parallelization to make a more efficient code which resulted in a faster code. In future research we could explore more areas where we could improve the efficiency of the code, and also do more runs for each board sizes to get mean values. We also implemented the Monte Carlo method of DLA and visualized the resulting clusters. If we had more time, it would be interesting to test out different number of walkers to see the effects on the cluster size to compare with different $p_s$ values. For the Gray-Scott model we also discovered that with no perturbation in the initial concentration we can get more symmetrical patterns. In future research, different boundary conditions and more parameters could be tested.

## References

[1] Gileesa McCormack. *Boxplot - SOR iteration for each $\eta$*. URL: https://github.com/Gileesa/ScientificComputing/blob/main/Assignment2/Figures/2.1/summary/boxplot_by_eta.pdf.

[2] Gileesa McCormack. *DLA Simulation gifs*. URL: https://github.com/Gileesa/ScientificComputing/tree/main/Assignment2/Figures/2.1/gif.

[3] Gileesa McCormack. *figures with different ps values*. URL: https://github.com/Gileesa/ScientificComputing/tree/main/Assignment2/Figures/2.2.

[4] Gileesa McCormack. *Gray-Scott Simulation gifs*. URL: https://github.com/Gileesa/ScientificComputing/tree/main/Assignment2/Figures/2.3.